TRELLO · VALIDATION SDK PYTHON · ITERATION 3

Aivana Python SDK Validation

I tested the official `aivana` Python SDK the way an outside developer would: by checking its behavior against the Aivana docs. This covers the Python part of Charter Rev2 Objective 1 / Deliverable 2 (CLI & SDK Validation).

SDK **aivana 0.3.0** (PyPI, latest) Python **3.10 (Linux) · 3.14 (macOS)** Live host **dev-developers.aivana.ai**

Final run **Oct 5, 2026** Status **Complete · ready for review**

64	50	14	0
test cases (40 offline, 24 live)	pass as documented	findings (2 high, 4 medium)	unexpected failures

The core of the SDK is solid. It builds requests correctly, keeps three-state options like `web_search=False`, refuses redirects so the API key stays on the configured host, and handles streaming and async. Live calls on the dev host confirmed the documented behavior for generation, streaming, async, Chat, the Intelligence Trace, JSON-schema answers and error classes. No model or provider names appeared in any trace or stream event. The problems are at the edges: **errors that don't match the docs**, `max_tokens` **not acting as a ceiling**, a **usage-reporting gap** on structured answers, and one **onboarding blocker** where the SDK's default host rejects the team's test key. Each finding has a test that reproduces it, so the suite will flag it once Aivana fixes it.

Findings

Severity reflects impact on an outside developer following the docs. "Repro" is the test ID in the suite. Offline repros use mocked HTTP. Live repros (`PY-LIV-…`) ran twice on Oct 5 with the team test key, and every live finding showed up in both runs.

F1 `High` A failed stream ends silently with a partial answer

What happens	When the API sends an `error` event mid-stream, the SDK passes it along as an ordinary chunk. The loop on the SDKs page (`for chunk in …: print(chunk.delta)`) prints the partial text and exits with no exception.
Docs say	SDKs page: "Every failure raises … a subclass of AivanaError." The Streaming page tells you to check for the error event, but the SDK examples never do.
Repro	`PY-STR-07`
Suggest	Raise the mapped `AivanaError` subclass when an `error` event arrives. If Aivana prefers not to, at least show error handling in the docs' streaming example.

F2 High Out of the box, the SDK sends test keys to a host that rejects them

What happens

`api_base` defaults to `https://developers.aivana.ai`, which answered our `ai_test_` key with 401 `invalid_api_key`. The same key worked on `dev-developers.aivana.ai`. A new developer's first call fails, and the error says the key is invalid or expired, which isn't the real cause.

Docs say

Quickstart: "The official SDKs already know where the Aivana API is, so there is no URL to configure."

Evidence

Confirmed live on Oct 5 (`PY-LIV-03`): same key, 401 on the default host and 200 on `dev-developers`. The Sept 29 first-call test showed the same with cURL and the Node SDK.

Suggest

Sponsor decision needed. Either issue test keys that work on the default host, or document `set_api_base()` / `AIVANA_API_BASE` for test keys and return a clearer 401 (for example, "test key used against production").

F5 Medium Network failures and timeouts escape as raw httpx exceptions

What happens

`httpx.ConnectError` and `httpx.ReadTimeout` are raised as-is. Code written to the docs (`except AivanaError`) crashes on a dropped connection, even though these are the failures the docs say to retry.

Repro

`PY-ERR-05`, `PY-ERR-06`

Suggest

Wrap transport errors in an `AivanaError` subclass (for example, a `ConnectionError` or `TimeoutError` class, or `code="network"`).

F8 Medium Retrying a failed `Chat.send()` sends the question twice

What happens

`Chat.send()` adds the user turn to the history before making the request. If the request fails, that turn stays in the history, so retrying `send()` with the same text puts it in the history twice. The extra turn costs tokens and can confuse the model.

Repro

`PY-CHT-04`

Suggest

Add the user turn to the history only after a successful response, or remove it if the request fails.

F11 Medium `max_tokens` isn't enforced as a ceiling

What happens

With `max_tokens=40`, the API returned 67 output tokens in the first run and 64 in the second, both billed and both with `finish_reason="stop"`, not `"length"`.

Docs say

README: "`max_tokens` is a ceiling, not a target. It can only shorten an answer."

Repro

`PY-LIV-11` (prompt: "Explain TCP vs UDP in detail.", `effort="low"`)

Suggest

Either enforce the cap and report `length` when it's hit, or document that `max_tokens` is a target the engine aims for, not a hard limit. A developer budgeting cost needs to know which.

F13 Medium Structured answers report zero output tokens

What happens	A successful `response_format` (JSON schema) call returned the validated object, but `usage.output_tokens` was `0` and credits covered input only. This happened in both runs.
Why it matters	Either output isn't being metered (a billing problem for Aivana), or it's metered but not reported (a cost-tracking problem for developers). The docs say a successful run is billed like any other.
Repro	`PY-LIV-22`, `PY-LIV-19`
Suggest	Ask Aivana which it is. Report output tokens for structured answers.

F3 Low Some 422s don't come back as `InvalidRequestError`

What happens	The SDK treats a 422 as `InvalidRequestError` only when its code is on a short allowlist of structured-output codes. Any other 422 (for example, a plain field-validation error) arrives as the base `AivanaError`, so `except InvalidRequestError` misses it.
Docs say	"InvalidRequestError: something in the request is wrong (400 or 422)."
Repro	`PY-ERR-03` (mocked). Live, a real bad field (`temperature=5`, `PY-LIV-21`) came back as 422 `invalid_request` and was classified correctly, with field details. So this is hardening against codes the API may add, not a current break.
Suggest	Map every 422 to `InvalidRequestError`.

F4 Low Production auth errors don't carry the request ID support asks for

What happens	On the dev host, a 401 includes `request_id` and the SDK passes it through. On the production host it doesn't, and the SDK reads the ID only from the body, never from the `X-Request-Id` header. So `err.request_id` is `None` there.
Docs say	Errors page: "request_id is always present." SDKs page: errors carry "the request id. Quote the request id when you contact support."
Repro	`PY-LIV-04` (production, fails) vs. `PY-LIV-02` (dev, passes). Header fallback: `PY-ERR-04`.
Suggest	API: include `request_id` on auth errors. SDK: fall back to the `X-Request-Id` header.

F6 Low `RateLimitError` drops the wait time

What happens	The API sends `retry_after_ms` and a `Retry-After` header, and the docs say to wait that long. The SDK exposes neither, so callers can't follow the docs without parsing the raw response.
Repro	`PY-ERR-07`
Suggest	Add a `retry_after` attribute in seconds to `RateLimitError`.

F7 `Low` A 504 isn't caught by the docs' retry example

What happens	Only 502 maps to `UpstreamError`. A 504 (which the docs say to retry), 413, 500 and 503 all become the base `AivanaError`, so the documented `except (RateLimitError, UpstreamError)` retry never catches a 504.
Repro	`PY-ERR-08`
Suggest	Map 504 and other 5xx responses to `UpstreamError`, or add a timeout error class.

F9 `Low · docs` The SDKs page "Configure" snippet crashes if the env var is missing

What happens	`aivana.api_key = os.environ["AIVANA_API_KEY"]` raises `KeyError` when the variable isn't set. The SDK already reads `AIVANA_API_KEY` on its own, so the line isn't needed.
Repro	`PY-CFG-07`
Suggest	Say in the docs that the env var is picked up automatically, or use `os.environ.get`.

F10 `Low · docs` Gaps between the docs and the README

What happens	`top_p` is supported and described in the SDK's docstrings, but it's missing from the SDKs page option list and from the README options table. The README's error list also leaves out `ForbiddenError`, which the SDKs page includes. Errors from `generate(stream=True)` only appear once you start iterating, and no page mentions this.
Suggest	Make one canonical options list and one error list, and use them in both the docs and the README.

F12 `Low` `output_shape="extract"` gives different shapes run to run

What happens	The same invoice prompt with `temperature=0` returned `{"response": "Thanks for sharing…"}` in the first run and properly extracted fields (`invoice_number`, `vendor`, `total_due`) in the second.
Repro	`PY-LIV-18` (evidence from both runs)
Suggest	The README already says `extract` doesn't guarantee a shape. Make that clearer in the docs and point developers who need fields to `response_format`, which worked reliably.

F14 `Low · docs` Every stream sends events the docs don't list

What happens	Even with `intelligence_trace` off, streams include `stage` events (for example "Reviewing multiple perspectives", "Resolving disagreements") and a `route` event (`{"model": "aivana-mmi", "provider": "aivana", "role": "mmi"}`). The `done` event also carries a `model` field. The Streaming page lists only `delta` , `done` , `error` and `trace` .
IP check	Passed. The payloads use Aivana branding only, and no underlying model or provider names appeared (`PY-LIV-35`). The stage names do describe the multi-perspective process, so Aivana should confirm they're intended as customer-facing.
Repro	`PY-LIV-34`
Suggest	Document `stage` and `route` , or send them only when the trace is requested.

Test matrix

Results from the final run on Oct 5 (macOS, Python 3.14, team test key, dev host). Running the suite again rewrites `results/matrix.md`.

ID	AREA	CHECK	MODE	RESULT
PY-INS-01	Install	Package installs from PyPI and reports its version	offline	PASS
PY-INS-02	Install	Runs on the documented minimum (Python ≥ 3.10)	offline	PASS
PY-INS-03	Install	Every name the docs use is importable from aivana	offline	PASS
PY-CFG-01	Configure	aivana.api_key = … sends X-API-Key	offline	PASS
PY-CFG-02	Configure	set_api_key() is equivalent to assignment	offline	PASS
PY-CFG-03	Configure	Falls back to AIVANA_API_KEY, read per request	offline	PASS
PY-CFG-04	Configure	A key set in code wins over AIVANA_API_KEY	offline	PASS
PY-CFG-05	Configure	No key: no header sent, 401 surfaces as AuthError	offline	PASS
PY-CFG-06	Configure	Default api_base is the documented host; trailing slash normalized	offline	PASS
PY-CFG-07	Configure	Docs configure snippet works when env var is unset	offline	DEFECT F9
PY-GEN-01	Generate	Parses a real dev-API response (answer, usage, finish_reason, notices)	offline	PASS
PY-GEN-02	Generate	POSTs /v1/generate with prompt; unset options omitted	offline	PASS
PY-GEN-03	Generate	Every documented option reaches the wire in snake_case	offline	PASS
PY-GEN-04	Generate	System prompt > 8000 chars rejected before any request	offline	PASS
PY-GEN-05	Generate	Attachments normalized (mimeType → mime_type); bad item rejected	offline	PASS
PY-GEN-06	Generate	response_format sent exactly as given; non-dict rejected	offline	PASS
PY-GEN-07	Generate	Tolerates new server fields (forward compatible)	offline	PASS
PY-GEN-08	Generate	Non-JSON 200 raises AivanaError, not JSONDecodeError	offline	PASS
PY-GEN-09	Generate	Redirects not followed; key never leaves api_base	offline	PASS
PY-GEN-10	Generate	generate_async returns GenerateResponse	offline	PASS
PY-GEN-11	Generate	messages= sent instead of prompt (manual history)	offline	PASS
PY-STR-01	Streaming	Delta chunks join into the answer; done carries usage	offline	PASS
PY-STR-02	Streaming	Streams POST /v1/generate:stream with the same body	offline	PASS
PY-STR-03	Streaming	json_schema + stream refused client-side with documented code	offline	PASS
PY-STR-04	Streaming	HTTP 401 on the stream raises AuthError	offline	PASS
PY-STR-05	Streaming	Async generate_stream yields the same chunks	offline	PASS
PY-STR-06	Streaming	Unknown and trace events pass through safely	offline	PASS

ID	AREA	CHECK	MODE	RESULT
PY-STR-07	Streaming	Mid-stream error event raises AivanaError	offline	DEFECT F1
PY-ERR-01	Errors	401/403/400/422/429/502/402 map to documented classes + codes	offline	PASS
PY-ERR-02	Errors	422 field details kept on the exception	offline	PASS
PY-ERR-03	Errors	Any 422 is an InvalidRequestError	offline	DEFECT F3
PY-ERR-04	Errors	request_id available on errors (header fallback)	offline	DEFECT F4
PY-ERR-05	Errors	Network failure raises an AivanaError subclass	offline	DEFECT F5
PY-ERR-06	Errors	Timeout raises an AivanaError subclass	offline	DEFECT F5
PY-ERR-07	Errors	RateLimitError exposes Retry-After	offline	DEFECT F6
PY-ERR-08	Errors	504 raises a retryable error class	offline	DEFECT F7
PY-CHT-01	Conversations	Chat sends history + previous_intent / pending_action	offline	PASS
PY-CHT-02	Conversations	Chat options apply every turn; send() overrides win	offline	PASS
PY-CHT-03	Conversations	reset() starts a fresh conversation	offline	PASS
PY-CHT-04	Conversations	A failed send() doesn't corrupt history	offline	DEFECT F8
PY-LIV-01	Auth	Invalid key → AuthError with code and message	live	PASS
PY-LIV-02	Auth	Errors carry request_id (dev host)	live	PASS
PY-LIV-03	Auth	Test key works with default api_base (no URL configured)	live	DEFECT F2
PY-LIV-04	Auth	Production-host errors carry request_id	live	DEFECT F4
PY-LIV-10	Generate	Basic generate returns answer, usage, finish_reason, notices	live	PASS
PY-LIV-11	Generate	max_tokens is a ceiling	live	DEFECT F11
PY-LIV-12	Generate	intelligence_trace returns steps and summary	live	PASS
PY-LIV-13	Generate	Trace stays customer-safe (no model/provider names)	live	PASS
PY-LIV-14	Generate	assistant_name honored	live	PASS
PY-LIV-15	Generate	messages= with a system message works	live	PASS
PY-LIV-16	Generate	web_search off → web_search_off notice	live	PASS
PY-LIV-17	Generate	stop_sequences marker not returned	live	PASS
PY-LIV-18	Generate	output_shape='extract' returns structured or structured_error	live	PASS
PY-LIV-19	Structured	json_schema → validated .structured (or not_available)	live	PASS
PY-LIV-20	Structured	Unsupported schema → invalid_response_schema	live	PASS
PY-LIV-21	Errors	temperature=5 → InvalidRequestError with details	live	PASS
PY-LIV-22	Structured	json_schema success reports output tokens	live	DEFECT F13

ID	AREA	CHECK	MODE	RESULT
PY-LIV-30	Streaming	Deltas arrive incrementally; done carries usage/notices	live	PASS
PY-LIV-31	Streaming	stream + trace emits trace events	live	PASS
PY-LIV-32	Streaming	Invalid key on stream → AuthError	live	PASS
PY-LIV-33	Async	generate_async and generate_stream end to end	live	PASS
PY-LIV-34	Streaming	Without trace, stream sends only documented events	live	DEFECT F14
PY-LIV-35	Streaming	Stream event payloads stay customer-safe	live	PASS
PY-LIV-40	Conversations	Chat remembers earlier turns	live	PASS

Questions for Aivana

1. Which host should test keys use, and should the default `api_base` work with them? (F2)
2. Should a mid-stream `error` event raise an exception, as the docs say every failure does? (F1)
3. Should we report findings as GitHub issues on `Aivana-Inc/aivana-sdk-python`, in Trello, or both? (Charter §19, item 3)
4. Is `max_tokens` meant to be a hard cap? (F11)
5. Are structured (JSON schema) answers meant to report zero output tokens? (F13)
6. Are the `stage` and `route` stream events intended for customers? (F14)

Next steps

- Team review of severities, then move the Trello card to Done with `results/` attached.
- File the findings wherever Aivana confirms, with repro test IDs. The SDK repo's GitHub Issues has no open issues yet.
- Node SDK card: F1, F3, F4 and F8 also appear in Node 0.7.0's source. Port the matrix IDs so the SDKs compare case by case.
- Rerun when Aivana ships 0.3.1+. Fixed findings show as "FIXED? (re-check)".

Rerun it

```
cd ~/Downloads/Aivana/sdk-validation-python
bash run_validation.sh                    # offline, free
export AIVANA_API_KEY=ai_test_...         # sponsor-approved test key
bash run_validation.sh                    # + 24 live cases on dev-developers.aivana.ai (~4 min, paced
for the rate limit)
```

Results: `results/matrix.md`, `matrix.csv`, and `evidence.jsonl` (keys redacted). Known-defect tests are strict expected failures: when the SDK fixes one, the run marks it "FIXED? (re-check)".